Chapter 13 Query Optimization

Query Optimization

- **Query optimization** is the process of ***selecting the most efficient query-evaluation plan*** from among the many strategies usually possible for processing a given query.
  - We do not expect ***users*** to write their queries so that they can be processed efficiently.
  - We expect the ***system*** to construct a ***query-evaluation plan*** that minimizes the cost of query evaluation. This is where ***query optimization*** comes into play.
  - The difference in ***cost*** between a ***good strategy*** and a ***bad strategy*** is often substantial. It is worthwhile to spend time on the selection of a good strategy.

Outline

- **Overview**
- **Transformation of Relational Expressions**
- **Estimating Statistics of Expression Results**
- **Choice of Evaluation Plans**

Overview

- Consider the query "***Find the names of all instructors in the Music department together with the course title of all the courses that the instructors teach***."
- We have the relational-algebra expression

$$\Pi_{name,title}(\sigma_{dept\_name = \text{"Music"}} (instructor \bowtie (teaches \bowtie \Pi_{course\_id,title}(course))))$$

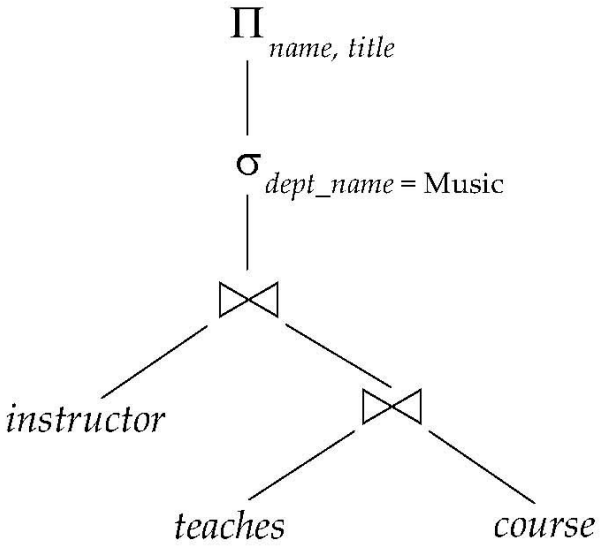
  - A large intermediate relation, $instructor \bowtie (teaches \bowtie \Pi_{course\_id,title}(course))$, is constructed.
  - In fact, we do not need to consider those tuples in the *instructor* relation that do not have ***dept_name = "Music"***.
  - By reducing the ***number of tuples*** of the *instructor* relation that we need to access, we can ***reduce the size of the intermediate result***.

Overview

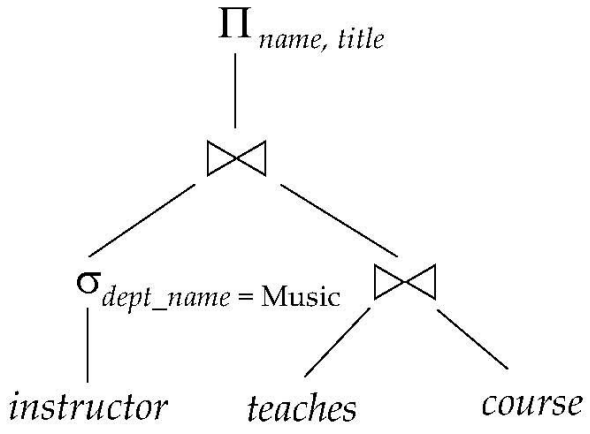
- Now we present the query as follow:

$\Pi_{name,title}((\sigma_{dept\_name="Music"}(\textbf{\textit{instructor}})) \bowtie (teaches \bowtie \Pi_{course\_id,title}(course)))$

  - It is equivalent to the initial expression, but generates ***smaller intermediate relations***.



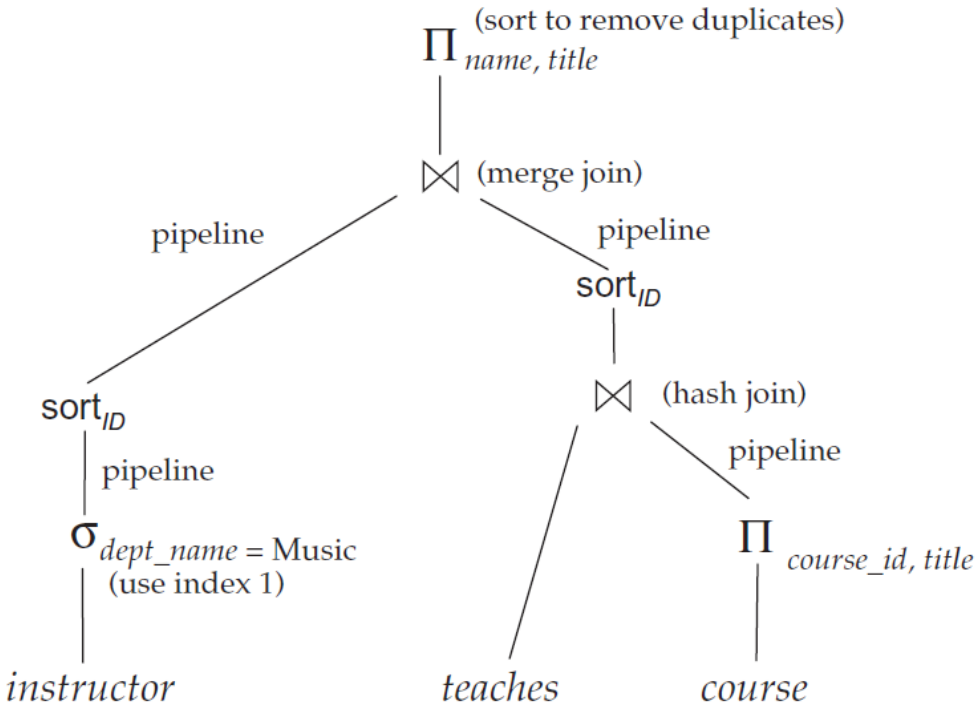
Initial expression tree



Transformed expression tree

Overview

- An **evaluation plan** defines exactly **what algorithm is used for each operation**, and **how the execution of the operations is coordinated**.



One possible evaluation plan for the previous transformed expression

- ***Hash join*** has been chosen for $teaches \bowtie \prod_{course\_id,\ title}(course)$ while the other join operations uses ***merge join***
- ***Sort*** *instructor* relations on *ID* which is the join attribute
- Where edges are marked with ***pipeline***, the output of the producer is pipelined directly to consumer without being written out to disk.

Overview

- Given a relational-algebra expression, the **query optimizer** comes up with a ***query-evaluation plan***, which computes the same result as the given expression and generate it in the ***least-costly way***.
- To find the ***least-costly query-evaluation plan***, the **optimizer** needs to generate alternative plans that produce the same result as the given expression, and to choose the least-costly one.

Overview (Cont.)

- ***Generation of query-evaluation plans*** involves three steps
  - ***Generating*** expressions that are ***logically equivalent*** to the given expression;
  - ***Annotating*** the resultant expressions in alternative ways to generate ***alternative query-evaluation plans***;
  - ***Estimating*** the cost of each evaluation plan, and ***choosing the least-costly one***.
- ***Equivalence rules*** specify ***how to transform an expression into a logically equivalent one***.

Transformation of Relational Expressions

- A query can be expressed in several different ways, with different ***costs*** of evaluation.
- If the two expressions generate the same set of tuples on ***every legal database instance***, two relational algebra expressions are said to be ***equivalent***.
  - A legal database instance is one that ***satisfies all the integrity constraints*** specified.
  - ***Order of tuples is irrelevant***. The two expressions may generate the tuples in different orders, but would be considered equivalent as long as the set of tuples is the same.

Equivalence Rules

- An ***equivalence rule*** says that expressions of two forms are equivalent, which means we can replace expression of first form by second, or vice versa.
  - Two expressions generate the same result on any legal database.
  - The optimizer uses equivalence rule to transform expressions into other logically equivalent expressions.
- We will list a number of ***equivalence rules*** on relational-algebra expressions.
  - ***Predicates***: θ, θ_1, θ_2…; ***lists of attributes***: L, L_1, L_2…; and ***relational-algebra expressions***: E, E_1, E_2…

Equivalence Rules

1. **Conjunctive selection operations** can be deconstructed into a sequence of individual selections (as a cascade of σ)

$$\sigma_{\theta 1 \wedge \theta 2}(E) = \sigma_{\theta 1}(\sigma_{\theta 2}(E))$$

2. **Selection operations are commutative**

$$\sigma_{\theta 1}(\sigma_{\theta 2}(E)) = \sigma_{\theta 2}(\sigma_{\theta 1}(E))$$

3. Only the final operation in **a sequence of projection operations** is needed; the others can be omitted (cascade of ∏)

$$\prod_{L1}(\prod_{L2}(\ldots(\prod_{Ln}(E))\ldots)) = \prod_{L1}(E)$$

4. **Selections** can be combined with **Cartesian products** and **theta joins**

   a. $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$ (definition of the theta join)

   b. $\sigma_{\theta 1}(E_1 \bowtie_{\theta 2} E_2) = E_1 \bowtie_{\theta 1 \wedge \theta 2} E_2$

Equivalence Rules

5. **Theta-join operations** (and **natural joins**) are **commutative**.

$$E_1 \bowtie_\theta E_2 = E_2 \bowtie_\theta E_1$$

- The equivalence does not hold if the ***order of attributes is taken into account***, but for simplicity we ignore the attribute order in most cases.
- The natural-join operator is simply a ***special case of the theta-join*** operator.

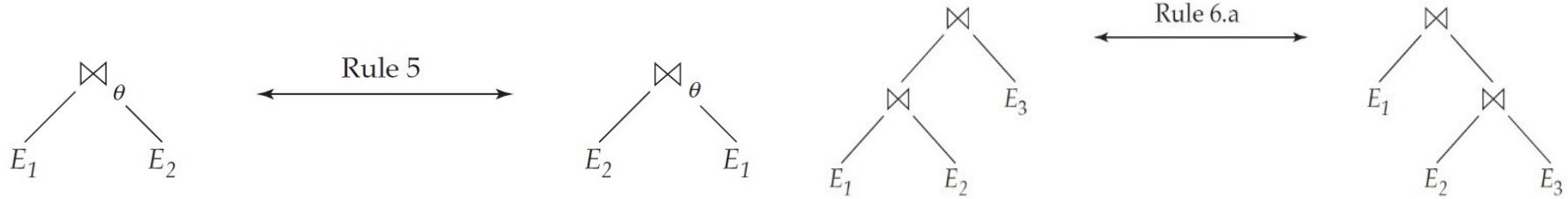
6. a) **Natural-join operations** are **associative.**

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

b) **Theta joins** are **associative** in the following manner:

$$(E_1 \bowtie_{\theta 1} E_2) \bowtie_{\theta 2 \wedge \theta 3} E_3 = E_1 \bowtie_{\theta 1 \wedge \theta 3} (E_2 \bowtie_{\theta 2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3.



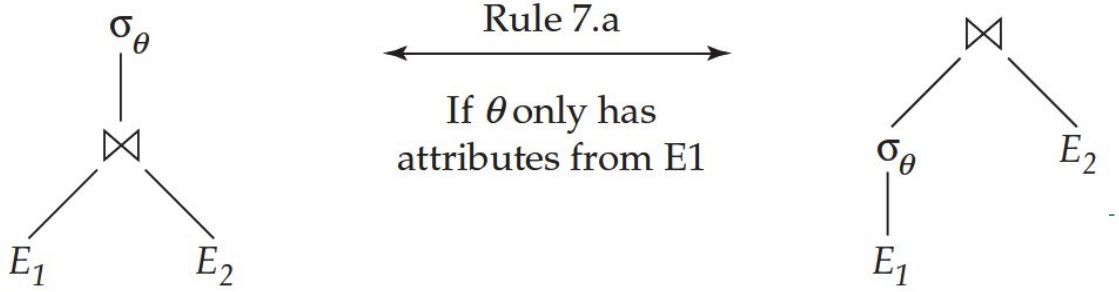
Equivalence Rules

7. The **selection operation** distributes over the **theta join operation** under the following two conditions
   a. It distributes when ***all the attributes in selection condition θ_0 involve only the attributes of one of the expressions*** (say, E_1) being joined.

$$\sigma_{\theta 0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta 0}(E_1)) \bowtie_{\theta} E_2$$

   b. It distributes when ***selection condition θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2.***

$$\sigma_{\theta 1 \wedge \theta 2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta 1}(E_1)) \bowtie_{\theta} (\sigma_{\theta 2}(E_2))$$



Equivalence Rules

8. The **projection operation** distributes over **the theta join operation** under the following conditions
   a. Let L_1 and L_2 be sets of attributes of E_1 and E_2, respectively. If θ involves only attributes from $L_1 \cup L_2$, then

$$\prod_{L1 \cup L2}(E_1 \bowtie_\theta E_2) = (\prod_{L1}(E_1)) \bowtie_\theta (\prod_{L2}(E_2))$$

   b. Consider a join $E_1 \bowtie_\theta E_2$.
      - Let L_1 and L_2 be sets of attributes from E_1 and E_2, respectively.
      - Let L_3 be attributes of E_1 that are involved in join condition θ, but are not in $L_1 \cup L_2$.
      - Let L_4 be attributes of E_2 that are involved in join condition θ, but are not in $L_1 \cup L_2$.

$$\prod_{L1 \cup L2}(E_1 \bowtie_\theta E_2) = \prod_{L1 \cup L2}((\prod_{L1 \cup L3}(E_1)) \bowtie_\theta (\prod_{L2 \cup L4}(E_2)))$$

Equivalence Rules

9. The **set operations union and intersection** are **commutative.**

$$E_1 \cup E_2 = E_2 \cup E_1$$

$$E_1 \cap E_2 = E_2 \cap E_1$$

- [Note] ***Set difference is not commutative***.

10. **Set union and intersection** are **associative**.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

Equivalence Rules

11. The **selection operation** distributes over the union, intersection, and set-difference operations.

$$\sigma_\theta (E_1 - E_2) = \sigma_\theta (E_1) - \sigma_\theta (E_2)$$

- The preceding equivalence, with $-$ replaced with either $\cup$ or $\cap$ holds. Further,

$$\sigma_\theta (E_1 - E_2) = \sigma_\theta (E_1) - E_2$$

- The preceding equivalence, with $-$ replaced with $\cap$ holds, but does **NOT** hold if $-$ replaced with $\cup$.

12. The **projection operation** distributes over the **union** operation.

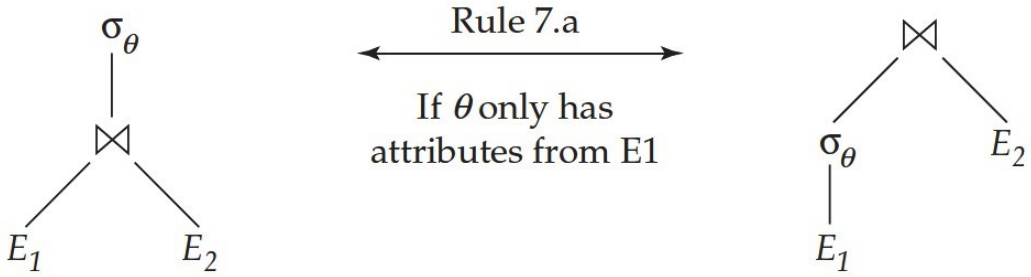
$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

Examples of Transformations

- Consider the query we mentioned before: "***Find the names of all instructors in the Music department together with the course title of all the courses that the instructors teach***."

$$\Pi_{name,title}\,(\sigma_{dept\_name\ =\text{“Music”}}\,(instructor \bowtie (teaches \bowtie \Pi_{course\_id,title}(course))))$$

θ_0 E_1 E_2



$$\Pi_{name,title}\,((\sigma_{dept\_name\ =\text{“Music”}}\,(instructor)) \bowtie (teaches \bowtie \Pi_{course\_id,title}(course)))$$

- Equivalent to our original algebra expression, but generates ***smaller intermediate relations***
- Note: the rule merely says that the two expressions are equivalent; it ***does not say that one is better than the other***.

Multiple transformations

- ***Multiple equivalence rules*** can be used, one after the other, on a query or on parts of the query.
- Consider the previous query, but restrict attention to ***instructors who have taught a course in 2009***. The new query is:

$$\Pi_{name,\ title}(\sigma_{dept\_name=\text{“Music”} \wedge year = 2009} (instructor \bowtie (teaches \bowtie \Pi_{course\_id,\ title}(course))))$$

  - We can first apply ***rule 6.a*** to this query:

$$\Pi_{name,\ title}(\sigma_{dept\_name=\text{“Music”} \wedge year = 2009} ((instructor \bowtie teaches) \bowtie \Pi_{course\_id,\ title}(course)))$$

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

  - Then apply ***rule 7.a***:

$$\Pi_{name,\ title}((\sigma_{dept\_name=\text{“Music”} \wedge year = 2009} (instructor \bowtie teaches)) \bowtie \Pi_{course\_id,\ title}(course))$$

$$\sigma_{\theta 0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta 0}(E_1)) \bowtie_{\theta} E_2$$

Multiple transformations

- Now consider the selection subexpression:

$\sigma_{dept\_name=\text{“Music”} \wedge year = 2009} (instructor \bowtie teaches)$

↓ **Rule 1:** $\sigma_{\theta 1 \wedge \theta 2}(E) = \sigma_{\theta 1}(\sigma_{\theta 2}(E))$

$\sigma_{dept\_name=\text{“Music”}} (\sigma_{year = 2009} (instructor \bowtie teaches))$

↓ **Rule 7.a:** $\sigma_{\theta 0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta 0}(E_1)) \bowtie_{\theta} E_2$ (twice)

$\sigma_{dept\_name=\text{“Music”}} (instructor) \bowtie \sigma_{year = 2009} (teaches)$

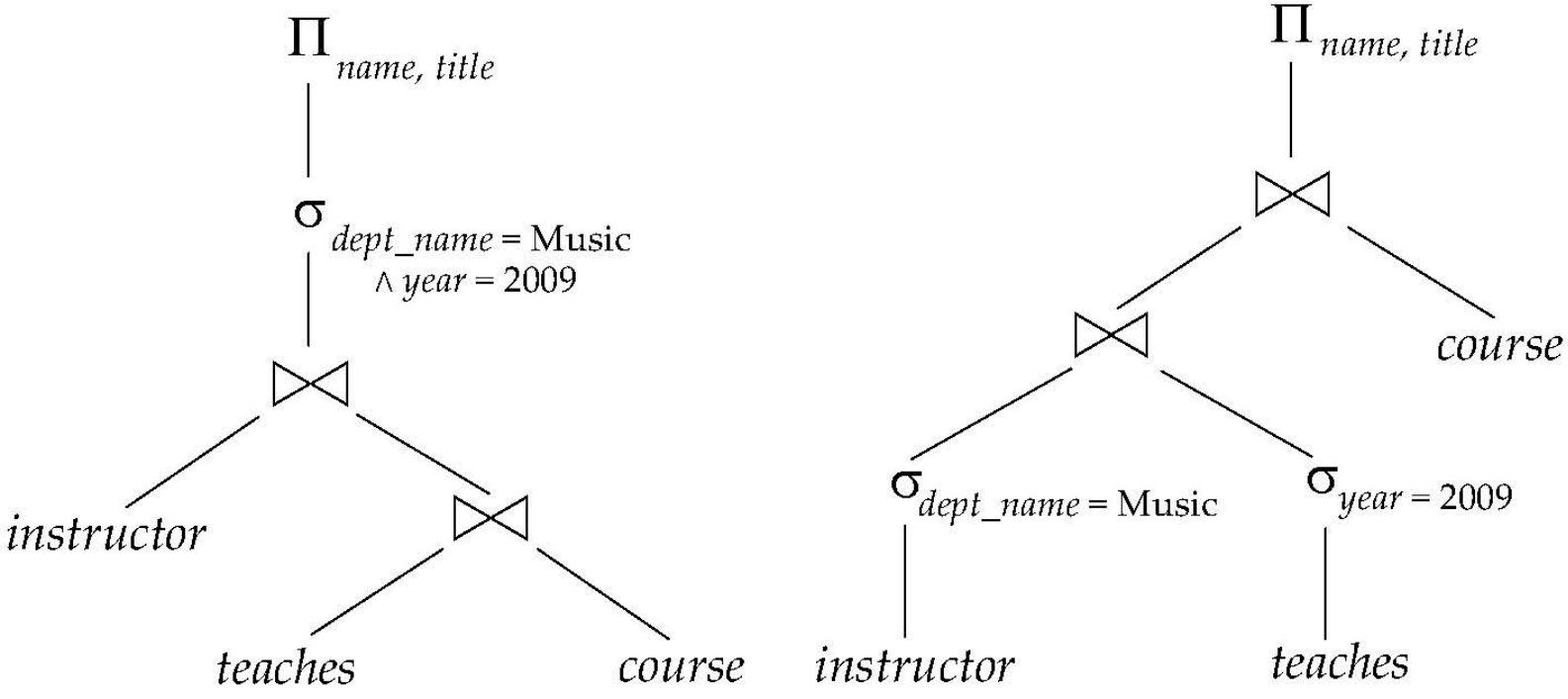
- Final expression can be equally obtained by using rule 7.b

$\sigma_{dept\_name=\text{“Music”} \wedge year = 2009} (instructor \bowtie teaches)$

↓ **Rule 7.b:** $\sigma_{\theta 1 \wedge \theta 2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta 1}(E_1)) \bowtie_{\theta} (\sigma_{\theta 2}(E_2))$

$\sigma_{dept\_name=\text{“Music”}} (instructor) \bowtie \sigma_{year = 2009} (teaches)$

Multiple transformations



(a) Initial expression tree

(b) Tree after multiple transformations

Multiple transformations

- If ***no rule can be derived from any combination of the others***, the set of equivalence rules is said to be ***minimal***.
  - An expression equivalent to the original expression may be ***generated in different ways***.
  - The number of different ways of generating an expression increases when we use a ***non-minimal set*** of equivalence rules.
  - ***Query optimizers*** therefore use minimal sets of equivalence rules.

Examples of Transformations

- Consider the expression

$\Pi_{name,\ title}((\sigma_{dept\_name=\text{“Music”}}(instructor) \bowtie teaches) \bowtie \Pi_{course\_id,\ title}(course))$

- Computing $\sigma_{dept\_name=\text{“Music”}}(instructor) \bowtie teaches$

  we obtain a relation whose schema is:

  (*ID*, ***name***, *dept_name*, *salary*, ***course_id***, *sec_id*, *semester*, *year*)

- By pushing projections based on 8.a and 8.b, we can eliminate unneeded attributes from intermediate results:

$\Pi_{name,\ title}((\Pi_{name,\ course\_id}(\sigma_{dept\_name=\text{“Music”}}(instructor) \bowtie teaches) \bowtie \Pi_{course\_id,\ title}(course))$

$\Pi_{L1 \cup L2}(E_1 \bowtie_\theta E_2) = (\Pi_{L1}(E_1)) \bowtie_\theta (\Pi_{L2}(E_2))$

$\Pi_{L1 \cup L2}(E_1 \bowtie_\theta E_2) = \Pi_{L1 \cup L2}((\Pi_{L1 \cup L3}(E_1)) \bowtie_\theta (\Pi_{L2 \cup L4}(E_2)))$

Join Ordering

- A ***good ordering of join operations*** is important for reducing the size of temporary results. Hence, most query optimizers pay a lot of attention to the ***join order***.
- Example: For all relations r_1, r_2, and r_3,

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

If $r_2 \bowtie r_3$ is quite large and $r_1 \bowtie r_2$ is small, we choose

$$(r_1 \bowtie r_2) \bowtie r_3$$

so that we compute and store a smaller temporary relation.

Join Ordering (Cont.)

- Consider the expression

$\Pi_{name,\ title}((\sigma_{dept\_name=\text{“Music”}}(instructor)) \bowtie teaches \bowtie \Pi_{course\_id,\ title}(course))$

  - We could compute $teaches \bowtie \Pi_{course\_id,\ title}(course)$ first, and then join result with $\sigma_{dept\_name=\text{“Music”}}(instructor)$.
    - The result of the first join is likely to be a large relation, since *it contains one tuple for every course taught*.
  - $\sigma_{dept\_name=\text{“Music”}}(instructor) \bowtie teaches$ is probably a small relation.
    - Only *a small fraction of the university's instructors are likely to be from the Music department*.
  - So it is better to compute $\sigma_{dept\_name=\text{“Music”}}(instructor) \bowtie teaches$ first. Then the temporary relation that we must store is smaller.

Join Ordering (Cont.)

- Other options to consider for evaluating our query.
- Consider the following relational-algebra expression:

  $(instructor \bowtie \prod_{course\_id,\ title}(course)) \bowtie teaches$

  - There are ***no attributes in common*** between

    $\prod_{course\_id,\ title}(course)$ and ***instructor***, which means the join is just a ***Cartesian product*** and produce ***a large temporary relation***.

  - Using the ***associativity*** and ***commutativity*** of the natural join, we can transform it to a more efficient expression:

    $(instructor \bowtie teaches) \bowtie \prod_{course\ id,\ title}(course)$

Enumeration of Equivalent Expressions

- **Query optimizers** use ***equivalence rules*** to ***systematically generate*** expressions equivalent to the given expression.
- Given a ***query expression E***, the set of ***equivalent expressions EQ*** initially contains only *E*. We can ***generate all equivalent expressions*** as follows:

  Repeat

  - ***apply all applicable equivalence rules*** on every sub-expression e_i of every equivalent expression E_i found so far
  - ***generates a new expression*** where e_i is transformed to match the other side of the rule
  - ***add newly generated expressions*** to the set of equivalent expressions *EQ*

  Until no more new expressions can be generated.

Enumeration of Equivalent Expressions

- This approach is **very expensive both in *space* and *time***. To reduce the cost, two approaches can be used:
  - Using ***expression-representation techniques*** that allow both expressions to point to shared sub-expressions to reduce the ***space requirement*** significantly
  - Not always necessary to generate every expression that can be generated with the equivalence rules. If an optimizer ***takes cost estimates of evaluation into account***, it may be able to ***avoid examining*** some of the expressions. We can thus reduce the time required for optimization.

Estimating Statistics of Expression Results

- The ***cost*** of an operation depends on the ***size*** and other ***statistics*** of its inputs.
  - Given an expression such as $a \bowtie (b \bowtie c)$ to estimate the cost of joining a with $(b \bowtie c)$, we need to have estimates of statistics such as the size of $b \bowtie c$.
- We will first list some ***statistics*** about database relations that are stored in database-system ***catalogs***, and then show ***how to use the statistics to estimate statistics on the results of various relational operations***.

Catalog Information

- The ***database-system catalog*** stores the following statistical information about database relations
  - n_r : ***number of tuples*** in the relation r
  - l_r : ***size of a tuple*** of relation r in bytes
  - f_r : ***blocking factor of r*** — i.e., the number of tuples of relation r that fit into one block
  - b_r : ***number of blocks*** containing tuples of r. When tuples of r are stored together physically in a file ($b_r = \lceil n_r / f_r \rceil$)
  - $V(A, r)$: ***number of distinct values*** that appear in r for attribute A; same as the size of $\prod_A(r)$. If A is a key for relation r, $V(A, r)$ is n_r
  - ***Statistics about indices***, such as the ***heights of B+-tree indices*** and ***number of leaf pages in the indices***

Catalog Information (Cont.)

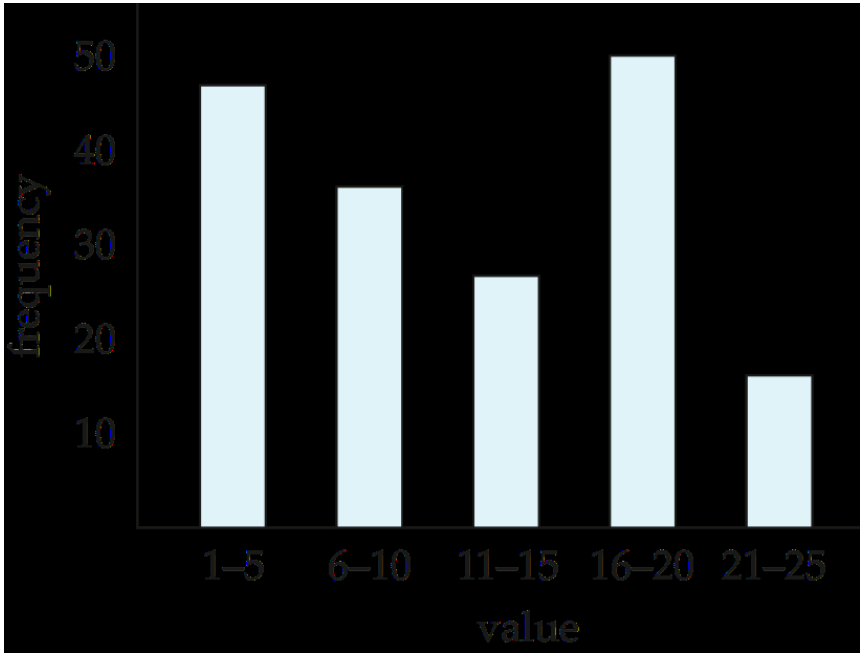
- For accuracy, every time a relation is modified, we must also update the statistic. However it causes large amount of ***overhead***.
- Most systems do not update the statistics on every modification. Instead, they update the statistics during periods of ***light system load***.
- If ***not too many updates occur in the intervals between the updates of the statistics***, the statistics will be sufficiently accurate to provide a good estimation of the relative costs of the different plans.

Histograms

- Real-world optimizers often maintain ***further statistical information*** to improve the accuracy of their cost estimates of evaluation plans.
- For instance, most databases store the ***distribution of values for each attribute*** as a **histogram**
  - In a histogram the ***values for the attribute*** are divided into a number of ***ranges***, and with each range the histogram associates the number of tuples whose attribute value lies in that range.
  - Histograms used in database systems usually ***record the number of distinct values in each range***, in addition to the number of tuples with attribute values in that range.

Histograms (Cont.)

- Several types of histograms
  - ***Equi-width histograms***: it divides the range of values into equal-sized ranges
  - ***Equi-depth histograms***: it adjusts the boundaries of the ranges such that each range has the same number of values.



Example: a histogram for an integer-valued attribute that takes values in the range 1 to 25.

Selection Size Estimation

- The size estimate of the result of a ***selection operation*** depends on the ***selection predicate***.
- We first consider a single equality predicate
  - **For $\sigma_{A=a}(r)$**
    - If we assume ***uniform distribution of values***, the selection result can be estimated to have $n_r/V(A, r)$ tuples (often not correct, like the *course_id* attribute in the *takes* relation).
    - If a ***histogram*** is available on attribute A, we can locate the range that contains the value a, and modify the above-mentioned estimate $n_r/V(A, r)$ by using the ***frequency count for that range instead of n_r***, and the ***number of distinct values that occurs in that range instead of $V(A, r)$***

Selection Size Estimation

- $\sigma_{A \le v}(r)$
  - Let ***c*** denote the estimated number of tuples satisfying the condition.
  - If the lowest and highest values for A, $min(A, r)$ and $max(A, r)$, are available in the catalog
    - $c = 0$, if $v < min(A, r)$
    - $c = n_r$, if $v \ge max(A, r)$
    - $c = n_r \cdot \frac{v - \min(A, r)}{\max(A, r) - \min(A, r)}$,otherwise
  - In absence of the value v, c is assumed to be $n_r/2$.

(*The case of $\sigma_{A \ge v}(r)$ is symmetric*)

Size Estimation of Complex Selections

- The **selectivity** of a condition θ_i is the ***probability that a tuple in the relation r satisfies*** θ_i .
- If s_i is the number of tuples satisfying θ_i in *r,* **selectivity** of θ_i is given by s_i / n_r .
- **Conjunction**: $\sigma_{\theta 1 \wedge \theta 2 \wedge \ldots \wedge \theta n}(r)$. Assuming that the conditions are ***independence***, the probability that a tuple satisfies all the conditions is simply the ***product of all these probabilities***. We estimate the number of tuples in the result is:

$$n_r * \frac{s_1 * s_2 * \ldots * s_n}{n_r^n}$$

Size Estimation of Complex Selections

- **Disjunction**: $\sigma_{\theta 1 \vee \theta 2 \vee \ldots \vee \theta n}(r)$. The ***probability*** that the tuple will satisfy the disjunction is then 1 minus the probability that it will satisfy ***none*** of the conditions. Then the estimated number of tuples is

$$n_r * \left(1 - (1 - \frac{s_1}{n_r}) * (1 - \frac{s_2}{n_r}) * \ldots * (1 - \frac{s_n}{n_r})\right)$$

- **Negation**: $\sigma_{\neg\theta}(r)$. In the absence of *null*s, the estimated number of tuples in $\sigma_{\neg\theta}(r)$ is estimated to be $n(r)$ minus the estimated number of tuples in $\sigma_{\theta}(r)$

$$n_r - size(\sigma_{\theta}(r))$$

Estimation of the Size of Joins

- The **Cartesian product** $r \times s$ contains $n_r * n_s$ tuples while each tuple occupies $(l_r + l_s)$ bytes. The ***estimated size*** is

$$n_r * n_s * (l_r + l_s)$$

- It is ***a little complicated to estimate the size of a*** **natural join**.
- Let r (R) and s(S) be relations and we will focus on the size of the ***natural join*** operation between these two relations.
- If $R \cap S = \varnothing$, that is, the relations have no attribute in common, then $r \bowtie s$ is the same as $r \times s$.

Estimation of the Size of Joins

- If ***R* ∩ *S* is a key for *R***, then a tuple of *s* will join with at most one tuple from *r*, therefore, the number of tuples in *r* ⋈ *s* is ***no greater than the number of tuples in s***.
  - Example: (***R***) *department* (***dept_name***, *building, budget*) ⋈ (***S***) *instrutctor* (*ID*, *name*, ***dept_name***, *salary*)
  - The case where *R* ∩ *S* is a key for *S* is symmetric.
- If ***R* ∩ *S* forms a foreign key in *S* referencing *R***, then the number of tuples in *r* ⋈ *s* is ***exactly the same as the number of tuples in s***.
  - Example: (***R***) *department* (***dept_name***, *building, budget*) ⋈ (***S***) *instrutctor* (*ID*, *name*, ***dept_name***, *salary*) [***foreign key*** (*dept_name*) reference *department*]

Estimation of the Size of Joins

- If $R \cap S = \{A\}$ **is not a key for neither** R **nor** S. We assume that ***each value appears with equal probability***. Then a tuple t of r will produce $n_s / V(A, s)$ tuples in $r \bowtie s$, since this number is the ***average number of tuples in s with a given value for the attributes A***.
  - Considering all the tuples in r, then the number of tuples in $r \bowtie s$ is estimated to be $n_r * n_s / V(A, s)$
  - Example: (***R***) *instrutctor* (*ID*, *name*, ***dept_name***, *salary*) $\bowtie$ (***S***) *course* (*course_id*, *title*, ***dept_name***, *credits*)
  - If we ***reverse the roles of r and s***, the estimate will be $n_r * n_s / V(A, r)$

Estimation of the Size of Joins

- The previous two estimates differ if $V(A, r) \neq V(A, s)$. If this situation occurs, there are likely to be ***dangling tuples*** that do not participate in the join, so the ***lower of these two estimates*** is probably the more ***accurate*** one.
- More important, the preceding estimate depends on the ***assumption that each value appears with equal probability***. More sophisticated techniques for size estimation have to be used if this assumption does not hold.

Join Size Estimation - Example

- Consider the expression ***student* ⋈ *takes*** and the following catalog information:
  - $n_{student} = 5{,}000$.
  - $f_{student} = 50$, which implies that $b_{student} = 5000/50 = 100$.
  - $n_{takes} = 10000$.
  - $f_{takes} = 25$, which implies that $b_{takes} = 10000/25 = 400$.
  - ***V*(*ID*, *takes*) = 2500**, which implies that on average, each student who has taken a course has taken 4 courses. (Attribute *ID* in *takes* is a foreign key referencing *student*.)
- In the example query ***student* ⋈ *takes***, *ID* in *takes* is a foreign key referencing *student*, and there is no *null* value on *take.ID*. Hence, the result has **exactly n_{takes} tuples**, which is **10000**.

Join Size Estimation - Example (Cont.)

- We can also compute the size estimates for ***student*** ⋈ ***takes*** without using information about foreign keys
  - ***V*(*ID*, *takes*) = 2500** and ***V*(*ID*, *student*) = 5000**
    - *ID* is the primary key for *student*.
  - The two estimates are 5000 * 10000/2500 = **20,000** and 5000 * 10000/5000 = **10000**

$$n_r * n_s / V(A, s) \qquad n_r * n_s / V(A, r)$$

  - We choose the ***lower estimate***, which in this case, is the same as our earlier computation using foreign keys.

Size Estimation for Other Operations

- **Projection**: The estimated size of a projection of the form $\Pi_A(r) = V(A, r)$, since projection eliminates duplicates.
- **Aggregation**: The estimated size of ${}_A\mathcal{G}_F(r) = V(A, r)$, since there is one tuple in ${}_A\mathcal{G}_F(r)$ for each distinct value of A.

Size Estimation for Other Operations

- **Set operations**
  - For **unions/intersections of selections on the same relation**: *rewrite* and *use* size estimate for selections
    - E.g. $\sigma_{\theta 1}(r) \cup \sigma_{\theta 2}(r)$ can be rewritten as $\sigma_{\theta 1 \vee \theta 2}(r)$
  - For **operations on different relations**, we estimate the sizes this way:
    - Estimated size of $r \cup s$ = ***size of r + size of s***.
    - Estimated size of $r \cap s$ = ***minimum of the sizes of r and s***.
    - Estimated size of $r - s$ = r.
    - All the three estimates may be ***inaccurate***, but provide ***upper bounds*** on the sizes.

Size Estimation for Other Operations

- **Outer join**:
  - Estimated size of $r \mathbin{⟕} s$ = *size of* $r \bowtie s$ + *size of* r
  - Case of right outer join is symmetric.

    Estimated size of

    $r \mathbin{⟖} s$ = *size of* $r \bowtie s$ + *size of* s
  - Estimated size of $r \mathbin{⟗} s$ = *size of* $r \bowtie s$ + *size of* r + *size of* s
  - All the three estimates may also be ***inaccurate***, but provide ***upper bounds*** on the sizes.

Estimation of Number of Distinct Values

- For **selections**, the ***number of distinct values of an attribute*** (or set of attributes) A in the result of a selection, $V(A, \sigma_\theta(r))$, can be estimated in these ways
  - If θ forces A to take a ***specified*** **value** (e.g., A=3), $V(A, \sigma_\theta(r)) = 1$
  - If θ forces A to take on one of a ***specified set of values*** (e.g., $(A = 1 \vee A = 3 \vee A = 4)$),
    $V(A, \sigma_\theta(r))$ = **the number of specified values**.
  - If θ is of the form ***A op v*** where op is a comparison operator, estimated $V(A, \sigma_\theta(r)) = V(A, r) * s$, where s is the ***selectivity*** of the selection.
  - In all the other cases: use an approximate estimate of $V(A, \sigma_\theta(r)) = \min(V(A, r), n_{\sigma\theta(r)})$

Estimation of Number of Distinct Values

- For **joins**, the **number of distinct values of an attribute** (or set of attributes) A in the result of a join, $V(A, r \bowtie s)$, can be estimated in these ways
  - If ***all attributes in A are from r***, estimated $V(A, r \bowtie s) = \min(V(A, r), n_{r \bowtie s})$
  - If ***all attributes in A are from s***, estimated $V(A, r \bowtie s) = \min(V(A, s), n_{r \bowtie s})$
  - If ***A contains attributes A1 from r and A2 from s***, then estimated $V(A, r \bowtie s) =$

$$\min(V(A1, r)*V(A2–A1, s), V(A1–A2, r)*V(A2, s), n_{r \bowtie s})$$

Note that some attributes may be in A1 as well as in A2, and A1−A2 and A2−A1 denote, respectively, attributes in A that are only from r and attributes in A that are only from s.

Estimation of Distinct Values (Cont.)

- Estimation of distinct values are straightforward for ***projections***. They are the same in $\Pi_{A\,(r)}$ as in r.
- The same holds for ***grouping attributes*** of ***aggregation***.
- For results of ***sum***, ***count***, and ***average***, we can assume, for simplicity, that ***all aggregate values are distinct***.
- For ***min***(A)and ***max***(A), the number of distinct values can be estimated as **min($V(A, r)$, $V(G, r)$)**, where G denotes the grouping attributes (considering $A=G$ or $A \neq G$).

Choice of Evaluation Plans

- Generation of expressions is only part of the query-optimization process.
- ***Each operation in the expression can be implemented with different algorithms***.
- An **evaluation plan** defines exactly
  - ***what algorithm should be used for each operation***
  - ***how the execution of the operations should be coordinated***
- A **cost-based optimizer** ***explores the space of all query-evaluation plans*** that are equivalent to the given query, and chooses ***the one with the least estimated cost.***

Cost-Based Join Order Selection

- For a complex join query, the number of different query plans that are equivalent to the query can be large.
- Example: find the ***best join-order for*** $r_1 \bowtie r_2 \bowtie \ldots r_n$
  - There are $(2(n-1))!/(n-1)!$ different join orders for above expression. E.g. with n = 3, there are 12 orderings

$r_1 \bowtie (r_2 \bowtie r_3)$	$r_1 \bowtie (r_3 \bowtie r_2)$	$(r_2 \bowtie r_3) \bowtie r_1$	$(r_3 \bowtie r_2) \bowtie r_1$
$r_2 \bowtie (r_1 \bowtie r_3)$	$r_2 \bowtie (r_3 \bowtie r_1)$	$(r_1 \bowtie r_3) \bowtie r_2$	$(r_3 \bowtie r_1) \bowtie r_2$
$r_3 \bowtie (r_1 \bowtie r_2)$	$r_3 \bowtie (r_2 \bowtie r_1)$	$(r_1 \bowtie r_2) \bowtie r_3$	$(r_2 \bowtie r_1) \bowtie r_3$

  - With $n = 7$, the number is 665280, with $n = 10$, the number is greater than ***17.6 billion***!

Cost-Based Join Order Selection

- No need to generate all the join orders. Using ***dynamic programming***, the ***least-cost join order*** for any subset of $\{r_1, r_2, \ldots r_n\}$ is ***computed only once and stored for future use***.
- For example, suppose we want to find the ***best join order*** of the form: $(r_1 \bowtie r_2 \bowtie r_3) \bowtie r_4 \bowtie r_5$
  - 12 different join orders for computing $r_1 \bowtie r_2 \bowtie r_3$
  - 12 orders for computing the join of the above result with r_4 and r_5
  - Once we have found the best join order for the subset of relations $\{r_1, r_2, r_3\}$, we can ignore all other orders. Then ***only* 12 + 12** choices need to be examined instead of 144.

Dynamic Programming in Optimization

- **To find best join tree for a set of *n* relations**
  - To find best plan for a set S of n relations, consider all possible plans of the form: $S_1 \bowtie (S - S_1)$ where S_1 is any non-empty subset of S.
  - ***Recursively compute*** costs for joining subsets of S to find the cost of each plan. Choose the cheapest.
  - When plan for any subset is computed, ***store it and reuse it*** when it is required again, instead of recomputing it.

Join Order Optimization Algorithm

```
procedure FindBestPlan(S)
    if (bestplan[S].cost ≠ ∞)
            return bestplan[S]
    // else bestplan[S] has not been computed earlier, compute it now
    if (S contains only 1 relation)
            set bestplan[S].plan and bestplan[S].cost based on the best way
            of accessing S  /* Using selections on S and indices on S */
    else for each non-empty subset S1 of S such that S1 ≠ S
            P1= FindBestPlan(S1)
            P2= FindBestPlan(S - S1)
            A = best algorithm for joining results of P1 and P2
            cost = P1.cost + P2.cost + cost of A
            if cost < bestplan[S].cost
                    bestplan[S].cost = cost
                    bestplan[S].plan = "execute P1.plan; execute P2.plan;
                                        join results of P1 and P2 using A"
    return bestplan[S]
```


Time complexity: $O(3^n)$

Interesting Sort Orders

- The ***order in which tuples are generated*** by the join of a set of relations is also important for finding the ***best overall join order***, since it can affect the cost of further joins (for instance, if ***merge join*** is used).
- An ***interesting sort order*** is a particular sort order of tuples that could be ***useful for a later operation***
  - For instance, generating the result of $r_1 \bowtie r_2 \bowtie r_3$ sorted on the ***attributes common with*** r_4 or r_5 may be useful, but sorted on attributes common to only r_1 and r_2 is not useful
  - Using ***merge join*** for computing $r_1 \bowtie r_2 \bowtie r_3$ may be ***costlier*** than using some other join technique, but it may provide an output sorted in an interesting sort order.

Interesting Sort Orders (Cont.)

- Hence, it is ***NOT sufficient*** to ***find the best join order for each subset of the set of n given relations***. Instead, we have to find the ***best join order*** for ***each subset***, for ***each interesting sort order of the join result for that subset***
  - The ***number of subsets of n relations*** is 2^n and the ***number of interesting sort orders*** is generally not large. Thus, about $O(2^n)$ join expressions need to be stored.
  - The ***dynamic programming*** algorithm for finding the best join order can be easily extended to handle sort orders. The cost of the extended algorithm depends on the number of ***interesting sort orders for each subset of relations***; since this number has been found to be small in practice, the cost remains at $O(3^n)$

Interesting Sort Orders (Cont.)

- Hence, it is ***NOT sufficient*** to ***find the best join order for each subset of the set of n given relations***. Instead, we have to find the ***best join order*** for ***each subset***, for ***each interesting sort order of the join result for that subset***
  - With $n = 10$, this number is around ***59,000***, which is much better than the ***17.6 billion*** different join orders.
  - More important, the ***storage required*** is much less than before, since we need to ***store only one join order for each interesting sort order*** of each of 1024 subsets of $r_1, \ldots, r_{10}$.
  - Although both numbers still increase rapidly with n, commonly occurring joins usually have less than 10 relations, and can be handled easily.

Heuristic Optimization

- Cost-based optimization is expensive, even with ***dynamic programming***.
  - The number of ***different evaluation plans*** for a query can be very large
  - Finding the ***optimal plan*** from this set requires a lot of computational effort
- System (**optimizers**) may use ***heuristics*** to reduce cost of optimization.

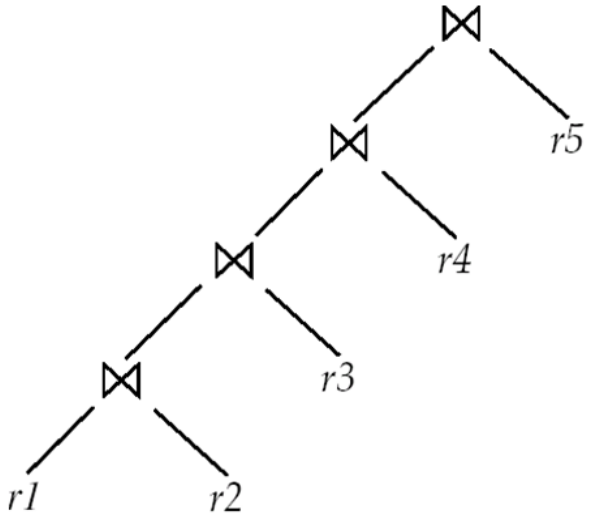
Heuristic Optimization (Cont.)

- ***Heuristic optimization*** transforms the ***query tree*** by using ***a set of rules*** that typically (but not in all cases) improve ***execution performance***
  - ***Perform selection early*** (reduces the number of tuples)
  - ***Perform projection early*** (reduces the number of attributes)
  - ***Perform most restrictive selection and join operations*** (i.e. with smallest result size) before other similar operations.
  - Some systems use only heuristics, others combine heuristics with partial cost-based optimization

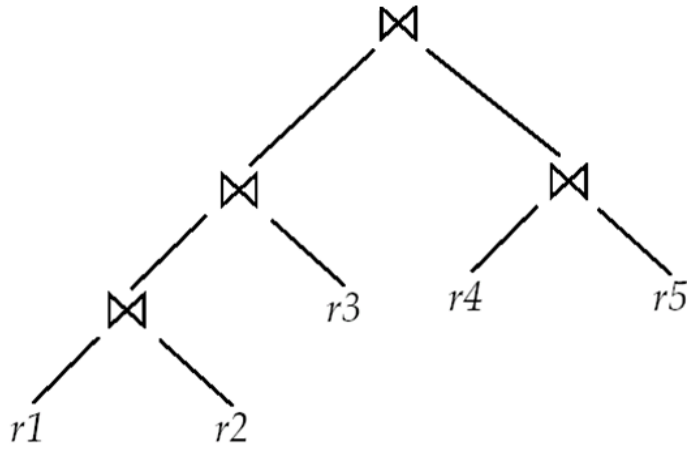
Heuristic Optimization (Cont.)

- Many optimizers considers only those join orders where the ***right operand of each join is one of the initial relations*** $r_1, \ldots, r_n$. Such join orders are called ***left-deep join orders***.
- It is convenient for ***pipelined evaluation***, since the right operand is a stored relation, and thus only one input to each join is pipelined.

Left Deep Join Trees



(a) Left-deep join tree



(b) Non-left-deep join tree

In ***left-deep join trees***, the right-hand-side input for each join is a relation, not the result of an ***intermediate join***.

- The time it takes to consider ***all left-deep join orders*** is ***O(n!)***.
- With the use of ***dynamic programming optimizations***, system can find the ***best join order*** in time ***$O(n*2^n)$*** which is contrasted with the ***$O(3^n)$*** time required to find the ***best overall join order***.

Heuristic Optimization (Cont.)

- Most optimizers allow a ***cost budget*** to be specified for query optimization.
  - The search for the optimal plan is terminated when the ***optimization cost budget*** is exceeded
  - The best plan found up to that point is returned.
- The budget itself may be set ***dynamically***
  - ***If a cheap plan is found*** for a query, the budget may be ***reduced***, on the premise that there is no point spending a lot of time optimizing the query if the best plan found so far is already quite cheap.

Heuristic Optimization (Cont.)

- Most optimizers allow a ***cost budget*** to be specified for query optimization.
  - The search for the optimal plan is terminated when the ***optimization cost budget*** is exceeded
  - The best plan found up to that point is returned.
- The budget itself may be set ***dynamically***
  - ***If the best plan found so far is expensive***, it makes sense to ***invest more time in optimization***, , which could result in a significant reduction in execution time.
  - So, optimizers usually ***first apply cheap heuristics*** to find a plan, and ***then start full cost-based optimization*** with a budget based on the heuristically chosen plan.

Heuristic Optimization (Cont.)

- Many applications ***execute the same query repeatedly***, but with ***different values for the constants***. (e.g., a university application may repeatedly execute a query to find the courses for which a student has registered, but each time for a different student with a different *ID*)
- Many optimizers optimize a query once, and ***cache the query plan*** (***plan caching***)
  - Whenever the query is executed again, perhaps with ***new values for constants***, the cached query plan is reused.
  - The optimal plan for the ***new constants may differ from the optimal plan for the initial values***, but as a heuristic the cached plan is reused.

Heuristic Optimization (Cont.)

- Even with the use of heuristics, ***cost-based query optimization*** imposes a ***substantial overhead***.
  - But the ***added cost of cost-based query optimization*** is usually less than offset by the ***saving at query-execution time***, which is dominated by slow disk accesses.
  - In some cases, one query can be ***optimized once***, and the selected query plan can be ***used each time the query is executed***.
  - So it is ***worth*** for expensive queries and most commercial systems include relatively sophisticated optimizers.

Homework

13.4 Consider the relations $r_1(A, B, C)$, $r_2(C, D, E)$, and $r_3(E, F)$, with primary keys A, C, and E, respectively. Assume that r_1 has 1000 tuples, r_2 has 1500 tuples, and r_3 has 750 tuples. Estimate the size of $r_1 \bowtie r_2 \bowtie r_3$, and give an efficient strategy for computing the join.

13.5 Consider the relations $r_1(A, B, C)$, $r_2(C, D, E)$, and $r_3(E, F)$ of Practice Exercise 13.4. Assume that there are no primary keys, except the entire schema. Let $V(C, r_1)$ be 900, $V(C, r_2)$ be 1100, $V(E, r_2)$ be 50, and $V(E, r_3)$ be 100. Assume that r_1 has 1000 tuples, r_2 has 1500 tuples, and r_3 has 750 tuples. Estimate the size of $r_1 \bowtie r_2 \bowtie r_3$ and give an efficient strategy for computing the join.